

Capítulo 15: Modelo de Actores: Pekko Fundamentos y Patrones

1. Explicación Teórica: El Modelo de Actores para la Concurrencia Masiva

El **Modelo de Actores** es un paradigma de concurrencia diseñado para simplificar la creación de sistemas distribuidos, concurrentes y tolerantes a fallos, abordando las dificultades históricas de la gestión manual de hilos y bloqueos en entornos de la JVM.

Scala es el lenguaje de implementación de muchos *frameworks* importantes, incluyendo Akka, que históricamente popularizó este modelo. Tras un cambio de licencia de Akka a un modelo comercial en 2022, la comunidad *open source* creó **Apache Pekko** como un *fork* de la última versión bajo licencia Apache 2.0, garantizando el acceso a este modelo de programación.

Principios Fundamentales del Actor Model

1. **Aislamiento y Estado Privado:** Un **actor** es una unidad de computación ligera y autónoma que encapsula su propio estado interno y comportamiento. Los actores **no comparten memoria mutable**. Esta **inmutabilidad por defecto** es clave, ya que elimina toda una clase de problemas complejos y propensos a errores relacionados con la corrupción de datos y las condiciones de carrera (*race conditions*) en sistemas concurrentes.
2. **Comunicación Asíncrona:** Los actores se comunican exclusivamente enviándose **mensajes asíncronos** a un buzón dedicado (*mailbox*). El actor receptor procesa los mensajes uno a la vez en secuencia, lo que previene cuellos de botella y garantiza que el sistema siga siendo reactivo.
3. **Tolerancia a Fallos y Supervisión:** Los actores se organizan en una **jerarquía**. Si un actor hijo falla (por una excepción inesperada), el fallo se contiene y se notifica a su actor **supervisor** (padre). El supervisor implementa una estrategia predefinida (*Supervision Strategy*): reanudar, reiniciar (borrando el estado interno) o detener permanentemente el actor. Esta es la filosofía **"Let-It-Crash"** (dejar que falle), que permite sistemas auto-recuperables.

Pekko Typed (Actores Tipados)

La versión moderna de Pekko (Pekko Typed) utiliza el riguroso sistema de tipos de Scala para que los actores sean **seguros en tipos**. El tipo de mensaje que un actor acepta está explícitamente declarado en su definición (`Behavior[Command]`), y el compilador garantiza que solo se le envíen mensajes de ese tipo.

2. Sintaxis y Patrones Básicos

En Pekko Typed, un actor se define por su **comportamiento** (`Behavior`), no por una clase que implementa un método genérico `receive`. El comportamiento define cómo debe reaccionar el actor a los mensajes.

A. Definición del Protocolo (Mensajes)

La forma idiomática es utilizar un **Tipo de Suma (ADT)** para definir todos los comandos posibles que un actor puede recibir, utilizando un `sealed trait` (o `enum` en Scala 3) como tipo base, y `case class` para cada mensaje. Esto garantiza la exhaustividad en el *pattern matching*.

```
1  // 1. Definición del ADT para los comandos que el actor acepta
2  object ActorDeTareas {
3      sealed trait Comando
4
5      // Mensaje con datos y un 'replyTo' para la respuesta
6      final case class AñadirTarea(tareaId: String, descripcion: String,
7                                  replyTo: ActorRef[Respuesta]) extends
8          Comando
9
10     // Mensaje sin datos, solo la acción
11     case object ObtenerEstado extends Comando
12
13     sealed trait Respuesta // ADT para posibles respuestas
14     final case class TareaAñadida(tareaId: String)
```

B. Definición del Actor (Behavior)

El comportamiento inicial de un actor se construye típicamente usando `Behaviors.setup` (para inicialización o *context access*) o `Behaviors.receiveMessage`.

```
1  import org.apache.pekko
2  import pekko.actor.typed.scaladsl.{Behaviors, ActorContext}
3  import pekko.actor.typed.{ActorRef, Behavior}
4
5  object ActorDeTareas {
6      import ActorDeTareas._ // Importar los comandos definidos arriba
7
8      // El estado interno inmutable del actor (Modelado con una case class)
9      case class Estado(tareas: Map[String, String])
10
11      // La función de fábrica que define el comportamiento
```

```

12     def apply(): Behavior[Comando] =
13         Behaviors.setup { context: ActorContext[Comando] =>
14             context.log.info("Actor de Tareas iniciado.")
15             // Inicia con un estado de tareas vacío
16             manejarTareas(Estado(Map.empty))
17         }
18
19     // Comportamiento funcional: El estado es un parámetro inmutable (FP
20     style)
21     private def manejarTareas(estado: Estado): Behavior[Comando] =
22         Behaviors.receiveMessage { mensaje: Comando =>
23             mensaje match {
24                 case AñadirTarea(id, desc, responderA) =>
25                     context.log.info("Recibido comando AñadirTarea: {}", id)
26
27                     // Crea un nuevo estado inmutable
28                     val nuevoEstado = estado.copy(tareas = estado.tareas + (id ->
29     desc))
30
31                     // Responde al remitente (necesita el replyTo)
32                     responderA ! TareaAñadida(id)
33
34                     // Devuelve el nuevo comportamiento con el estado actualizado
35                     manejarTareas(nuevoEstado)
36
37                 case ObtenerEstado =>
38                     context.log.info("Estado actual: {} tareas", estado.tareas.size)
39                     // Si no cambia el estado o comportamiento, se devuelve
40                     Behaviors.same
41             }
42         }
43     }

```

C. Comunicación y Referencias

La comunicación se realiza mediante el operador `!` (bang o *tell*), y es asíncrona. Para un patrón de solicitud-respuesta (*request-response*), la petición debe incluir explícitamente una referencia de actor (`ActorRef[T]`) a la que enviar la respuesta, como `replyTo`.

```

1 // Ejemplo de envío de un comando desde otro actor o desde el ActorSystem
2 val actorRef: ActorRef[ActorDeTareas.Comando] = ??? // Obtención de la
3 referencia
4 // Se crea una sonda (probe) para recibir la respuesta (Técnica de
5 testing/integración)

```

```

5  val sondaRespuesta: ActorRef[ActorDeTareas.Respuesta] = ???
6
7  // Envío asíncrono del mensaje:
8  actorRef ! ActorDeTareas.AñadirTarea("001", "Implementar módulo",
    sondaRespuesta)

```

3. Patrones Avanzados: Estado y Tolerancia a Fallos

A. Manejo de Estado: Estilos Funcional vs. OO

Pekko permite dos estilos para manejar el estado interno de un actor:

1. **Estilo Funcional (FP Style):** El estado se pasa como un **parámetro inmutable** a la función que define el `Behavior`. Cuando el actor procesa un mensaje que modifica el estado, devuelve una llamada recursiva a la misma función (`manejarTareas(nuevoEstado)`), la cual se convierte en el **nuevo comportamiento** para el siguiente mensaje. **Esta es la forma preferida en Scala idiomático**, ya que es inherentemente inmutable.
2. **Estilo Orientado a Objetos (OO Style):** El estado se mantiene como un campo mutable (`var`) dentro de la clase que define el comportamiento. Al recibir un mensaje, el actor muta la variable (`var`) y devuelve `Behaviors.same`. Aunque es más familiar para los desarrolladores de Java, el encapsulamiento del actor garantiza que sigue siendo *thread-safe* porque solo un mensaje se procesa a la vez.

B. Tolerancia a Fallos y Jerarquía (Supervisión)

La jerarquía de actores es el mecanismo de **tolerancia a fallos** de Pekko. El actor padre (supervisor) es el responsable de decidir cómo manejar los fallos del hijo. El

`Behaviors.supervise` se utiliza para envolver el comportamiento del actor y aplicar la estrategia.

```

1  import pekko.actor.typed.SupervisorStrategy
2  import scala.concurrent.duration._
3
4  // 1. Definición del actor hijo que puede fallar
5  object ServicioHijo {
6      def apply(): Behavior[String] = Behaviors.setup { context =>
7          context.log.info("Servicio Hijo iniciado.")
8          Behaviors.receiveMessage {
9              case "fallar" => throw new RuntimeException("Error inesperado en
    servicio.")
10             case _ => Behaviors.same
11          }
12      }

```

```

13 }
14
15 // 2. Actor Padre que supervisa y aplica una estrategia
16 object SupervisorServicio {
17     def apply(): Behavior[String] =
18         Behaviors.setup { context =>
19             // Estrategia: Reiniciar el actor hijo en caso de cualquier fallo
20             (Exception)
21             val estrategia = SupervisorStrategy.restart.withLimit(maxNrOfRetries
22             = 10,
23             withinTimeRange = 10.seconds)
24
25             val hijo = context.spawn(
26                 Behaviors.supervise(ServicioHijo()).onFailure[Exception]
27                 (estrategia),
28                 name = "HijoSupervisado"
29             )
30
31             Behaviors.receiveMessage {
32                 case "pruebaFallo" => hijo ! "fallar"; Behaviors.same
33                 case _ => Behaviors.unhandled
34             }
35         }
36     }
37 }

```

En este ejemplo, si el `ServicioHijo` recibe "fallar", lanzará una excepción, pero el `SupervisorServicio` lo reiniciará hasta 10 veces en un período de 10 segundos, siguiendo la estrategia definida. El padre solo reinicia la instancia del hijo, manteniendo el resto del sistema en funcionamiento.

4. Comparativa con Java y Python

Aspecto	Java (Modelos Thread/Lock)	Python (Event Loop/Asyncio)	Scala (Pekko/Actor Model)
Modelo de Concurrency	Hilos pesados (OS <i>Threads</i>). Uso de <code>synchronized</code> y <code>Lock</code> para sincronización.	Bucle de eventos, <i>coroutines</i> (Python 3+).	Actores/Fibras Ligeras. Se ejecuta concurrencia masiva en un <i>pool</i> de hilos reducido.
Manejo de Estado Compartido	Mutable. Propenso a <i>race conditions</i> y <i>deadlocks</i> .	Mutable por defecto.	Inmutable por defecto. El estado está encapsulado en el actor; no hay

Aspecto	Java (Modelos Thread/Lock)	Python (Event Loop/Asyncio)	Scala (Pekko/Actor Model)
			memoria compartida, eliminando bloqueos.
Manejo de Errores	Excepciones (<code>try/catch</code>) que interrumpen la ejecución del hilo.	Excepciones/manejo de errores.	Supervisión Jerárquica. Los fallos (excepciones inesperadas) se aíslan y se gestionan reactivamente mediante estrategias.
Protocolo de Comunicación	Llamadas a métodos sincrónicas.	Llamadas a funciones o métodos (usualmente asíncronos en Asyncio).	Mensajes Asíncronos (!). Requiere un <code>ActorRef</code> explícito (<code>replyTo</code>) para las respuestas, lo que garantiza el tipado.

5. Best Practices (*The Scala Way*)

- Modelar Comandos como ADTs:** Utiliza `sealed trait / enum` y `case class` para definir el protocolo de mensajes de tu actor. Esto garantiza que los mensajes sean **inmutables** y permite al compilador verificar si se cubren todos los casos en el *pattern matching* del actor.
- Estado Funcional por Defecto:** Aunque Pekko permite el uso de `var` encapsuladas, la forma idiomática y recomendada en Scala es definir el comportamiento como una **función recursiva que acepta y devuelve el estado como un parámetro inmutable** (`Behavior[Comando] = manejarTareas(nuevoEstado)`).
- Comunicación Explícita:** Nunca confíes en un `sender()` implícito. Si el actor debe responder, el mensaje de solicitud (`Command`) debe incluir explícitamente el campo `replyTo: ActorRef[Respuesta]` , asegurando la seguridad de tipos para la respuesta.
- Diseño Jerárquico para la Resiliencia:** Estructura la aplicación en una jerarquía lógica de actores. Define la **Supervision Strategy** (`Behaviors.supervise`) en el actor padre para gestionar los fallos de sus hijos (ej. `SupervisorStrategy.restart`), aplicando el principio "Let-It-Crash".
- Cluster Sharding para Escalabilidad:** Para sistemas distribuidos que gestionan muchas entidades con estado (como sesiones de usuario o entidades DDD), utiliza **Cluster Sharding** para distribuir transparentemente los actores por el clúster, sin que el remitente necesite conocer su ubicación física.

El Actor Model es una **fábrica de resiliencia**: en lugar de intentar evitar que las máquinas fallen, asume que fallarán y diseña un sistema que puede reiniciar instantáneamente las unidades de trabajo más pequeñas (actores) bajo la estricta vigilancia de un supervisor, protegiendo la integridad del sistema completo.